

DDoS Attack Detection Using Machine Learning

1. Project Title

DDoS Attack Detection Using Machine Learning and Flask Web Application

2. Abstract

Distributed Denial of Service (DDoS) attacks are a major cybersecurity threat in which a target system or network is overwhelmed with a large volume of malicious traffic. Such attacks can reduce system availability and affect the performance of network services.

This project presents a **Machine Learning-based DDoS Attack Detection System** that analyses network traffic data and classifies traffic into two categories: **BENIGN** and **DDoS**. The system uses two supervised machine learning algorithms, **Random Forest** and **Support Vector Machine (SVM)**, to detect malicious network traffic. The predictions from both models are combined using a voting mechanism to produce the final classification.

The trained models are integrated into a **Flask-based web application**, allowing users to upload a CSV file containing network traffic features. The application processes the uploaded data, performs feature scaling, generates predictions, and displays the results through graphical visualizations such as a **pie chart and bar chart**.

The project achieved high performance, with the Random Forest model obtaining approximately **99.99% accuracy**, while the combined model achieved approximately **97.67% accuracy** with an F1 score of approximately **97.997%**.

3. Introduction

With the rapid growth of Internet-based services, network security has become an important concern. One of the most common and dangerous network attacks is the **Distributed Denial of Service (DDoS) attack**.

A DDoS attack attempts to make a network service unavailable by generating a large amount of traffic toward a target. Traditional rule-based detection techniques may have difficulty identifying changing attack patterns. Machine Learning provides an alternative approach by learning patterns from previously collected network traffic data.

In this project, Machine Learning algorithms are trained using network traffic features to distinguish between normal and DDoS traffic. The project combines **Random Forest and SVM classifiers** and provides a web-based interface for practical prediction.

4. Problem Statement

The objective of this project is to develop an automated system capable of identifying whether network traffic is **BENIGN** or associated with a **DDoS attack**.

The system should:

- Accept network traffic data in CSV format.
- Extract relevant network traffic features.
- Preprocess and scale the input data.
- Apply trained Machine Learning models.
- Classify traffic as **BENIGN** or **DDoS**.
- Display prediction results visually.
- Allow users to download prediction results.

5. Objectives

The main objectives of this project are:

1. To develop a Machine Learning-based DDoS detection system.
2. To preprocess network traffic data for Machine Learning.
3. To train a **Random Forest classifier**.
4. To train an **SVM classifier**.
5. To combine the predictions of both models.
6. To evaluate the models using accuracy, precision, recall, and F1 score.
7. To develop a Flask-based web interface.
8. To allow users to upload network traffic CSV files.
9. To display prediction results using charts.
10. To provide an easy-to-use interface for DDoS detection.

6. Technologies Used

Programming Language

- Python

Machine Learning

- Scikit-learn
- Random Forest
- Support Vector Machine (SVM)
- MinMaxScaler

Data Processing

- Pandas
- NumPy

Visualization

- Matplotlib

Web Development

- Flask
- HTML/CSS templates

Model Storage

- Joblib

Development Environment

- Python 3.12
- PyCharm / Visual Studio Code
- Web browser

The project's dependency file includes Flask 3.0.3, NumPy 1.26.4, Pandas 2.2.2, Scikit-learn 1.5.1, Matplotlib 3.9.2, Joblib 1.4.2, and other supporting packages.

7. Dataset

The project uses the network traffic dataset:

Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv

The dataset contains network traffic information and a **Label** column used for classification.

The labels are converted into numerical values:

- **BENIGN → 0**
- **DDoS → 1**

The training script loads the dataset and selects a set of network traffic features for model training.

8. Features Used

The system uses various network traffic characteristics, including:

- Source Port
- Destination Port
- Protocol
- Flow Duration
- Total Forward Packets
- Total Backward Packets
- Total Length of Forward Packets
- Total Length of Backward Packets
- Forward Packet Length
- Backward Packet Length
- Flow IAT
- Forward IAT
- Backward IAT
- Packet Length
- FIN Flag Count
- SYN Flag Count
- RST Flag Count

- PSH Flag Count
- ACK Flag Count
- URG Flag Count
- CWE Flag Count
- ECE Flag Count

These features represent different characteristics of network communication and are used by the Machine Learning models to identify traffic patterns.

9. Data Preprocessing

Before training the Machine Learning models, the dataset is prepared through several steps.

9.1 Feature Selection

The required network traffic columns are selected from the dataset.

9.2 Label Encoding

The original labels are converted into numerical values:

BENIGN → 0

DDoS → 1

9.3 Train-Test Split

The dataset is divided into training and testing datasets.

The project uses:

Training Data = 70%

Testing Data = 30%

with random_state=42 for reproducibility.

9.4 Feature Scaling

The project uses **MinMaxScaler** to scale the input features before training.

The scaler is fitted on the training data and then applied to the testing data.

10. Machine Learning Models

10.1 Random Forest Classifier

Random Forest is an ensemble Machine Learning algorithm that combines multiple decision trees to produce a classification result.

In this project, the Random Forest classifier is configured with:

Number of estimators = 100

Random State = 42

The model is trained on the scaled training data and used to predict whether network traffic is BENIGN or DDoS.

Advantages

- High classification performance.
- Works well with many features.
- Handles complex relationships.
- Less prone to overfitting compared with a single decision tree.

11. Support Vector Machine

The second classification algorithm used is **Support Vector Machine (SVM)**.

The project uses:

Kernel = RBF

Gamma = auto

The SVM model is trained using the scaled network traffic data.

SVM attempts to find a decision boundary that separates different classes of network traffic.

12. Combined Model

To improve the detection system, the predictions generated by Random Forest and SVM are combined.

The project uses the following voting mechanism:

$\text{combined_pred} = (\text{rf_pred} + \text{svm_pred}) \geq 1$

This means that if either classifier predicts the traffic as DDoS, the combined prediction is classified as DDoS.

The combined numerical prediction is then converted into the final labels:

1 → DDoS

0 → BENIGN

The Flask application performs the same combination when processing uploaded files.

13. Model Evaluation

The models are evaluated using four major performance metrics:

Accuracy

Accuracy measures the percentage of correctly classified samples.

Precision

Precision measures how many samples predicted as DDoS were actually DDoS.

Recall

Recall measures how many actual DDoS samples were correctly detected.

F1 Score

F1 score provides a balance between precision and recall.

14. Model Performance

The obtained results are:

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	99.991%	99.996%	99.988%	99.992%
SVM	97.648%	96.079%	99.953%	97.978%
Combined Model	97.670%	96.077%	99.996%	97.998%

These values are taken directly from the project's recorded model evaluation results.

15. Isolation Forest Evaluation

The project also contains an evaluation result for an **Isolation Forest** model.

Its recorded performance was significantly lower:

- Accuracy: approximately **38.05%**
- Precision: approximately **0.045%**
- Recall: approximately **0.0039%**
- F1 Score: approximately **0.0071%**

Based on the project's evaluation file, Isolation Forest was therefore not used as part of the final DDoS detection system. Instead, Random Forest and SVM were used as the combined detection model.

16. Flask Web Application

The Machine Learning models are integrated into a **Flask web application**.

The application loads the trained Random Forest and SVM models using Joblib.

The Flask application provides an interface where users can upload a CSV file containing network traffic data.

The uploaded CSV file is read using Pandas:

CSV File



Pandas DataFrame



Feature Selection



MinMax Scaling



Random Forest + SVM



Combined Prediction



BENIGN / DDoS

The application performs these operations automatically when the user uploads a file.

17. Prediction Process

The complete prediction process is:

Step 1: Upload CSV

The user uploads a network traffic CSV file through the web interface.

Step 2: Read Data

The Flask application reads the uploaded file using Pandas.

Step 3: Feature Extraction

The predefined network traffic features are extracted.

Step 4: Feature Scaling

The data is scaled using MinMaxScaler.

Step 5: Random Forest Prediction

The scaled data is passed to the trained Random Forest model.

Step 6: SVM Prediction

The data is also passed to the trained SVM model.

Step 7: Combine Predictions

The predictions of both models are combined.

Step 8: Generate Result

Each record is classified as:

BENIGN

or

DDoS

Step 9: Visualization

The application generates a pie chart and a bar chart representing the prediction results.

18. Web Application Output

The developed web application provides graphical representations of the prediction results.

18.1 Prediction Pie Chart

The pie chart represents the percentage distribution between:

- BENIGN traffic
- DDoS traffic

In the submitted output screenshot, the prediction distribution is approximately:

DDoS : 59.1%

BENIGN : 40.9%

The pie chart is generated dynamically by the Flask application using Matplotlib.

Figure Caption

Figure 1: Prediction Results – Pie Chart showing the distribution of BENIGN and DDoS traffic.

19. Prediction Bar Chart

The application also generates a bar chart showing the number of predictions for each class.

The screenshot shows two categories:

- DDoS
- BENIGN

The DDoS category has a higher number of predictions than BENIGN in the submitted test output.

The application creates this chart by counting the predicted labels and plotting the corresponding values.

Figure Caption

Figure 2: Prediction Counts – Bar chart showing the number of DDoS and BENIGN predictions.

20. User Interface

The web application provides the following functionality:

- CSV file upload
- Automated network traffic analysis
- DDoS prediction
- BENIGN prediction
- Prediction visualization
- Pie chart generation
- Bar chart generation
- CSV prediction download
- Upload another file option

The project instructions specify running `app.py`, opening the local Flask URL in a browser, and uploading the test file through the web interface.

21. Model Saving

After training, the project saves the trained models using **Joblib**.

The following files are generated:

`random_forest_model.pkl`

`svm_model.pkl`

`scaler.pkl`

These saved models can then be loaded by the Flask application instead of retraining the models every time the application starts.

22. System Architecture

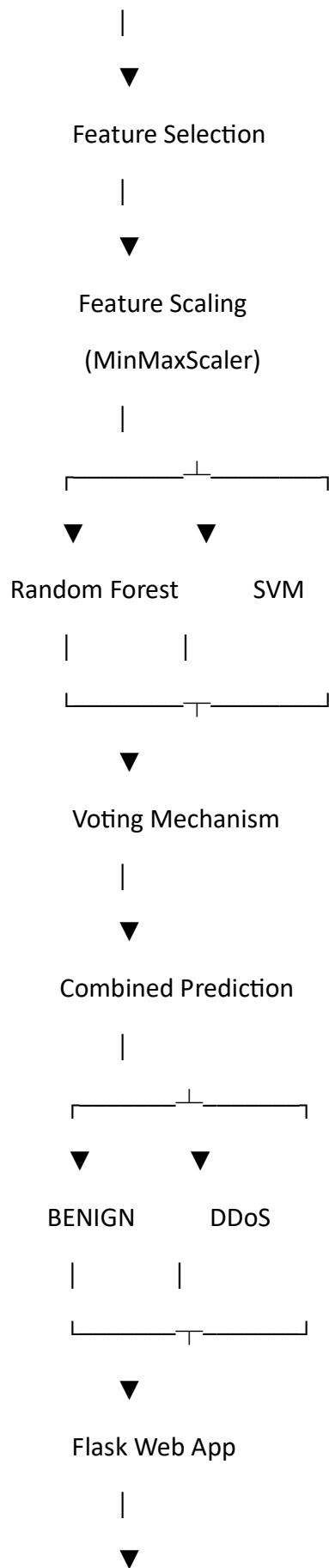
You can include the following architecture in your Word document:

Network Traffic Dataset

|



Data Preprocessing



23. Project Workflow

The complete workflow of the project can be summarized as:

1. Collect network traffic dataset.
2. Load the dataset using Pandas.
3. Select relevant network traffic features.
4. Convert labels into numerical values.
5. Split the dataset into training and testing sets.
6. Scale the features using MinMaxScaler.
7. Train the Random Forest model.
8. Train the SVM model.
9. Generate predictions.
10. Combine Random Forest and SVM predictions.
11. Evaluate model performance.
12. Save trained models using Joblib.
13. Develop Flask web application.
14. Upload test CSV through the browser.
15. Perform real-time prediction.
16. Display prediction results using charts.

24. Advantages of the System

The proposed system has several advantages:

- Provides automated DDoS detection.
- Uses Machine Learning instead of only rule-based detection.
- Uses multiple classification algorithms.
- Provides high Random Forest classification performance.
- Provides a user-friendly web interface.

- Supports CSV-based network traffic analysis.
- Generates visual prediction results.
- Saves trained models for later use.
- Reduces the need for manually analyzing network traffic.

25. Limitations

The current implementation has some limitations:

1. The system primarily works with CSV-based network traffic data.
2. The prediction quality depends on the quality and characteristics of the training dataset.
3. The system is designed for the traffic classes represented in the dataset, particularly BENIGN and DDoS.
4. The combined model does not outperform the Random Forest model on the recorded test metrics.
5. The Isolation Forest evaluation in the project shows very poor performance and therefore it was not selected for the final detection process.

26. Future Scope

The project can be improved in the future by:

- Adding more types of network attacks.
- Using larger and more diverse datasets.
- Implementing real-time network traffic monitoring.
- Improving the ensemble/voting strategy.
- Testing additional Machine Learning algorithms.
- Using Deep Learning models for complex traffic patterns.
- Adding an alert mechanism for detected attacks.
- Creating a database for storing detection results.
- Adding authentication to the web application.
- Deploying the application on a cloud server.

- Developing a dashboard for continuous network monitoring.

27. Conclusion

This project successfully demonstrates the application of **Machine Learning for DDoS attack detection**. Network traffic data is processed and classified using **Random Forest and Support Vector Machine algorithms**.

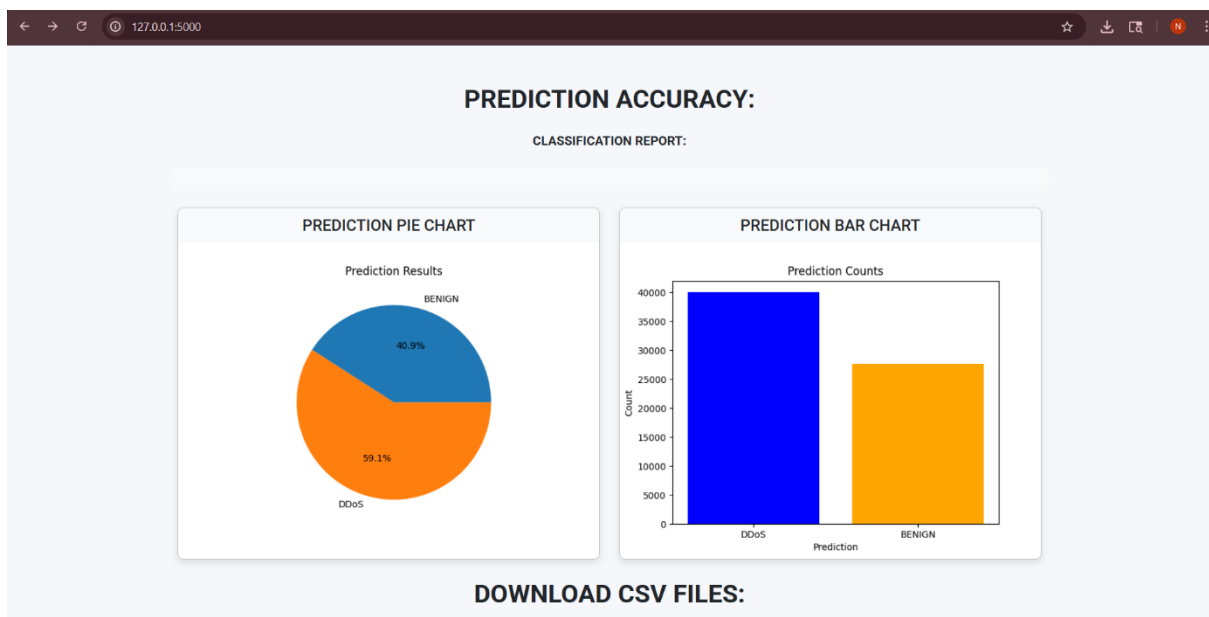
The Random Forest model achieved approximately **99.99% accuracy**, while SVM achieved approximately **97.65% accuracy**. The combined model achieved approximately **97.67% accuracy**, with a recall of approximately **99.996%**, demonstrating its ability to identify a very high proportion of DDoS traffic in the recorded evaluation.

The trained models were integrated into a Flask web application, making the system easier to use. Users can upload a CSV file containing network traffic information and receive classification results in the form of **BENIGN or DDoS**, together with graphical visualizations.

Overall, the project demonstrates how Machine Learning, data preprocessing, model evaluation, and web application development can be combined to create a practical cybersecurity-oriented DDoS detection system.

28. Outputs

Screenshot 1 – DDoS Prediction Result



Description:

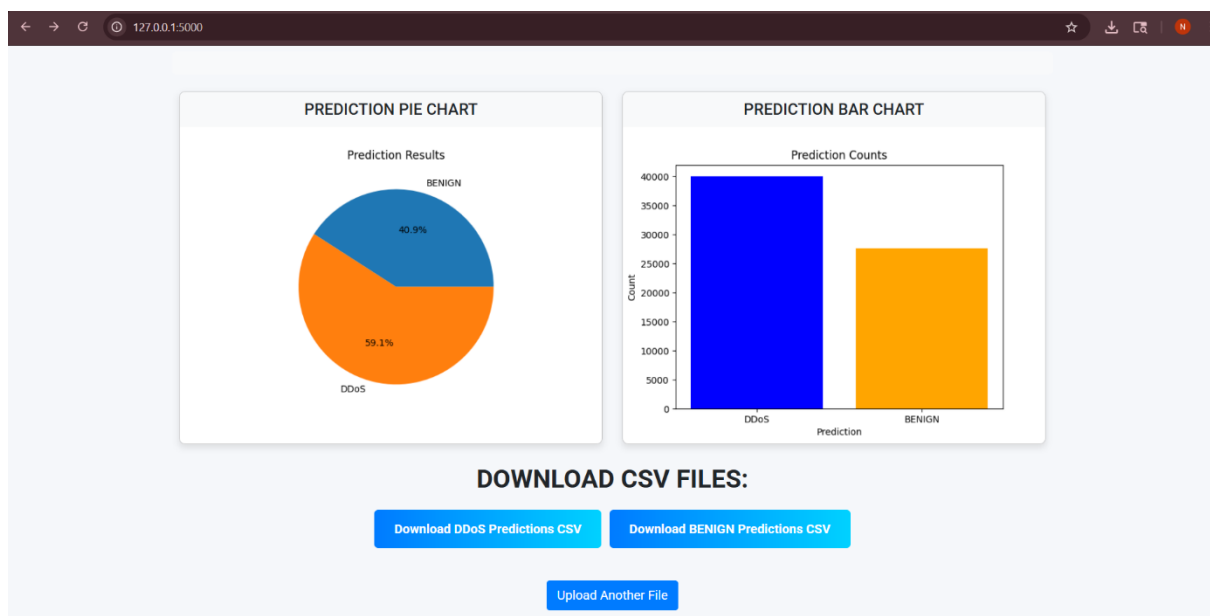
The screenshot shows the prediction results generated by the Flask web application. The pie chart represents the percentage distribution of BENIGN and DDoS traffic, while the bar chart displays the prediction count for each class.

Observed Result:

- DDoS: **59.1%**
- BENIGN: **40.9%**

Figure Caption:

Figure 3: DDoS Detection Prediction Results – Pie Chart and Bar Chart

Screenshot 2 – Prediction Accuracy and Results**Description:**

The second screenshot demonstrates the prediction results page of the developed Flask application. The interface provides graphical representations of the classification results and options to download the generated prediction CSV files.

Figure Caption:

Figure 4: Flask Web Application – Prediction Results and CSV Download Interface

29. References / Project Files

The project consists of the following important components:

anomaly_detection/

|

|— model/

| |— random_forest_model.pkl

| |— svm_model.pkl

| |— scaler.pkl

|

|— static/

| |— Prediction charts

|

|— templates/

| |— ddos.html

| |— results.html

|

|— app.py

|— model_train.py

|— test.csv

|— model_accuracy.txt

|— requirements.txt

|— readme.txt

The README describes the intended setup flow: create a Python environment, install the dependencies, run app.py, open the local Flask URL, and upload the test file.